LECTURE 8

Transformers, Intuitively

Instructor Manual

An analogy-first walk-through: meaning as coordinates, words that change with context, attention as a radio net, and how the encoder and decoder share the work

AK DHAMIJA, Researcher & Instructor, AI & Machine Learning

dhamija.ak@gmail.com • 9811453163

Unit 4: Simulation Strategies in Warfare — AI & Machine Learning in Warfare

This manual contains: the 45-minute session plan with timings; a slide-by-slide teaching guide for all 33 slides; key messages, facilitation tips, anticipated questions with model answers, misconceptions and in-class activities; live demonstrations with the interactive studio; consistency notes on the source material; the complete quiz answer key with rationale and scoring rules; and the assignment with marking scheme and full model solutions.

Contents

1 Lecture at a Glance **2**

2 Session Plan (45 Minutes) **3**

3 Slide-by-Slide Teaching Guide **3**

3.1 1. Overview (slides 2–4, about 3 min) . . . 4
3.2 2. Meaning as Numbers (slides 5–7, about 3 min) . . . 4
3.3 3. Static vs Contextual (slides 8–10, about 3 min) . . . 4
3.4 4. Encoder & Decoder (slides 11–13, about 3 min) . . . 4
3.5 5. Tokens & Positions (slides 14–16, about 3 min) . . . 4
3.6 6. Attention (slides 17–21, about 3 min) . . . 5
3.7 7. Multi-Head (slides 22–23, about 2 min) . . . 5
3.8 8. Completing the Block (slides 24–26, about 3 min) . . . 5
3.9 9. The Decoder (slides 27–29, about 3 min) . . . 5
3.10 10. Wrap-Up (slides 30–31, about 2 min) . . . 5
3.11 11. Summary (slides 32–33, about 2 min) . . . 6

4 Key Messages **6**

5 Facilitation Tips **6**

6 Anticipated Questions and Model Answers **6**

7 Common Misconceptions **7**

8 In-Class Activities and Discussion Prompts **7**

9 Run It Live: Studio Demonstrations **7**

10 Quiz Administration and Answer Key **8**

11 Assignment: Marking Scheme and Model Solutions **11**

11.1 Problem 1: Cosine similarity on a meaning map (20 marks) . . . 11
11.2 Problem 2: Static versus contextual embeddings (20 marks) . . . 12
11.3 Problem 3: One attention step by hand (20 marks) . . . 12
11.4 Problem 4: Encoder and decoder roles; masking (20 marks) . . . 13
11.5 Problem 5: Assessing a defence use (20 marks) . . . 13
11.6 Problem 6 (bonus): What two heads add (20 marks) . . . 14

1 Lecture at a Glance

Lecture	8 — Transformers, Intuitively
Duration	45 minutes: 5 min opening Q&A, 30 min teaching, 10 min auto-quiz
Slides	33 (Beamer PDF, 16:9), 11 sections
Prerequisite	Lecture 7; students should have attempted its quiz and assignment.
Student materials	`Lecture8_Slides_Beamer.pdf`; `Lecture8_Reference_Text.pdf`; `Lecture8_Quiz.html`; `Lecture8_Answers_Assignment.html` (release after the quiz)
Interactive studio	`Lecture8_Transformer_Intuition_Lab.html` (live demonstrations in Section 9)
Assessment	Auto-quiz: 15 MCQs, 40 s each, $+1/-0.25$. Assignment: 5 problems, 100 marks, plus a 20-mark bonus problem.

Overview

Lecture 8 builds intuition for Transformers before Lecture 9 supplies the full mathematics. It starts from what a language model does — predict a probability for every next token, then repeat — and explains embeddings as coordinates on a meaning map, measured by cosine similarity, with directions that carry meaning. The running example is the word "battery" (a unit of guns or a power cell): a static embedding cannot tell the senses apart, and one attention step, computed by hand with toy vectors, moves the same vector towards "artillery" in one sentence and towards "radio" in another. The lecture then completes the picture with analogies checked by arithmetic: queries, keys and values as a radio net; multi-head attention as a sensor suite; residual connections as margin notes; layer normalisation; the feed-forward network; the decoder writing one token at a time with a causal mask; and cross-attention. Defence uses and cautions (hallucination, prompt injection, tokenisation of jargon) close the session.

Learning outcomes

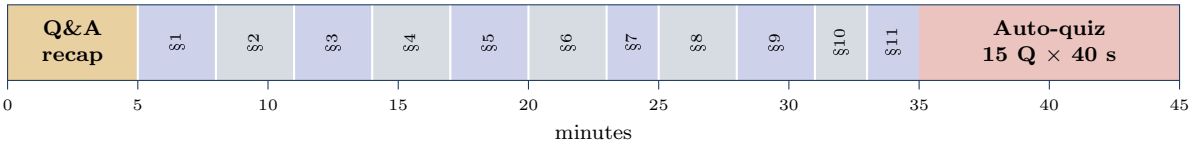
By the end of the lecture students should be able to:

1. Explain what a language model does and how it writes one token at a time.
2. Describe embeddings as coordinates on a meaning map and compute cosine similarity.
3. Explain why a static embedding fails for words with several senses and what a contextual embedding is.
4. Compute one attention step by hand and explain queries, keys and values.
5. Explain multi-head attention, residual connections, normalisation, the feed-forward layer, masking and cross-attention, and assess a defence use.

Syllabus mapping

- Deep learning for language and sequences — **Transformers** (intuitive treatment)

2 Session Plan (45 Minutes)



Clock	Slides	Segment	Min
0:00–5:00	—	**Opening Q&A and recap** (see prompts below)	5
5:00–8:00	2–4	1. Overview (3 slides)	3
8:00–11:00	5–7	2. Meaning as Numbers (3 slides)	3
11:00–14:00	8–10	3. Static vs Contextual (3 slides)	3
14:00–17:00	11–13	4. Encoder & Decoder (3 slides)	3
17:00–20:00	14–16	5. Tokens & Positions (3 slides)	3
20:00–23:00	17–21	6. Attention (5 slides)	3
23:00–25:00	22–23	7. Multi-Head (2 slides)	2
25:00–28:00	24–26	8. Completing the Block (3 slides)	3
28:00–31:00	27–29	9. The Decoder (3 slides)	3
31:00–33:00	30–31	10. Wrap-Up (2 slides)	2
33:00–35:00	32–33	11. Summary (2 slides)	2
35:00–45:00	—	**Auto-quiz** (instructions 1 min, 15 questions × 40 s = 10 min of answering; collect scores)	10

Slide 1 is the title slide and is shown while students settle. The teaching segment covers 32 content slides in 30 minutes, an average pace of 1.1 slides per minute.

Minute allocations are proportional to slide counts; shift time towards the sections students find hardest.

Opening Q&A prompts (first 5 minutes)

1. From Lecture 7: why does a CNN use far fewer parameters than a dense network on an image?
2. What does a residual connection $y = F(x) + x$ do for the gradient?
3. From Lecture 2: what does softmax do to a list of scores?

Take two or three answers per prompt; correct misconceptions briefly and note any topic that needs revisiting.

3 Slide-by-Slide Teaching Guide

Each subsection gives the slide range, the suggested time, what to emphasise, and the slide titles in order. Numbers refer to pages of the Beamer PDF.

3.1 1. Overview (slides 2–4, about 3 min)

Teaching note. Objectives; the next-token job with softmax probabilities (support 0.635); the journey from text to output.

2 Learning Objectives & Session Structure
3 What a Language Model Actually Does
4 The Journey Through This Lecture

3.2 2. Meaning as Numbers (slides 5–7, about 3 min)

Teaching note. ID numbers versus embeddings; the toy past-tense direction and retreat − advance + advanced ≈ retreated; where embeddings come from and their sizes (300 to 12,288).

5 Why a Simple ID Number Is Not Enough
6 Directions on the Meaning Map Carry Meaning
7 Where Embeddings Come From

3.3 3. Static vs Contextual (slides 8–10, about 3 min)

Teaching note. The battery problem; the running example's result table; long-range context and why recurrent networks struggled.

8 The "Battery" Problem: One Word, Several Meanings
9 Running Example: Context Pulls "Battery" Two Different Ways
10 Context Can Come From Far Away

3.4 4. Encoder & Decoder (slides 11–13, about 3 min)

Teaching note. Encoder and decoder roles; self-supervised training on next words; BERT, GPT and the original design compared.

11 Two Halves: The Encoder Reads, the Decoder Writes
12 Training vs Inference: Learning From Text That Labels Itself
13 BERT, GPT and the Original Design

3.5 5. Tokens & Positions (slides 14–16, about 3 min)

Teaching note. Tokens and IDs ([CLS] 101, [SEP] 102); embedding lookup as a row copy; adding a position vector because attention is parallel and order-blind.

14 Inside the Encoder, Step 1: Tokens and Token IDs
15 Step 2: Look Up Each Token's Vector
16 Step 3: Stamp Each Vector With Its Position

3.6 6. Attention (slides 17–21, about 3 min)

Teaching note. Attention weights over a sentence; the radio-net analogy; $q = xW^Q$, $k = xW^K$, $v = xW^V$; the four-step recipe and why K^T and $\sqrt{d_k}$; the worked battery arithmetic for sentences A and B.

17 Step 4: Attention — Which Words Should Update Which?
18 Query, Key, Value: The Radio-Net Analogy
19 Where Q, K and V Really Come From
20 The Recipe in Four Steps
21 Worked Example: The Arithmetic Behind the "Battery" Picture

3.7 7. Multi-Head (slides 22–23, about 2 min)

Teaching note. Heads as a sensor suite: BERT-base $12 \times 64 = 768$; what heads might track, and the caution about reading heat-maps.

22 Multi-Head Attention: A Sensor Suite, Not a Single Sensor
23 What Different Heads Might Track (Illustrative)

3.8 8. Completing the Block (slides 24–26, about 3 min)

Teaching note. Residuals as margin notes; layer normalisation of $(2, 4, 6, 8)$; the feed-forward network ($768 \to 3{,}072 \to 768$); one block, then a stack.

24 Residual Connections and Normalisation
25 The Feed-Forward Network: Each Token Thinks It Over
26 One Transformer Block — Then Stack It

3.9 9. The Decoder (slides 27–29, about 3 min)

Teaching note. Generation step by step; masked self-attention; cross-attention as the decoder consulting the report; the whole journey on one slide.

27 The Decoder Writes One Token at a Time
28 Cross-Attention: The Decoder Consults the Report
29 The Whole Journey on One Slide

3.10 10. Wrap-Up (slides 30–31, about 2 min)

Teaching note. Defence relevance (contextual meaning, structured reporting, search by meaning) and cautions; further exploration.

30 Defence Relevance and Cautions
31 Further Exploration

3.11 11. Summary (slides 32–33, about 2 min)

Teaching note. Summary and quick check; look-ahead to the technical lecture.

32 Summary and Quick Check
33 Looking Ahead — Lecture 9: Transformers & Attention (Technical)

4 Key Messages

Students should leave the room able to state these in their own words:

Key point

1. A language model predicts the next token; long answers are that one step repeated.
2. An embedding is a learned position on a meaning map; closeness is measured by direction (cosine).
3. Attention builds a contextual embedding: the word's vector plus a weighted blend of what its neighbours offer.
4. Multi-head attention, residuals, normalisation and a feed-forward network make one block; blocks stack.
5. Fluent output is not verified output; keep a human in the loop for anything that feeds a decision.

5 Facilitation Tips

- Write the battery table for sentence A on the board and have students compute sentence B before showing it; the weights 0.480, 0.206, 0.314 are quick with a calculator.
- In the Transformer Intuition Lab, sentence C ("radio battery fired") shows mixed context: battery is pulled half-way and stays ambiguous.
- Insist on separating analogy from mechanism: after each analogy, show the matrix operation it stands for.

6 Anticipated Questions and Model Answers

Check your understanding — anticipated question

Q: Does the model understand the sentence?

A: It computes contextual vectors that support accurate predictions. Whether that is understanding is a separate question; it does not verify facts.

Check your understanding — anticipated question

Q: Why not give each word a vector by hand?

A: Meaning depends on usage across billions of contexts; learning the vectors from data captures this, and the dimensions have no human labels.

Check your understanding — anticipated question

Q: Are attention weights an explanation?

A: They show where information was gathered in one layer and one head, not why an output was chosen; later layers mix the values again.

7 Common Misconceptions

Misconception	Correction
"The model looks facts up."	It predicts probable tokens; there is no database unless one is explicitly attached.
"Transformers read left to right."	The encoder reads all tokens in parallel; only decoders are causal.
"Twelve heads cost twelve times as much."	Each head works in d/h dimensions; the total is about one full-width head.

8 In-Class Activities and Discussion Prompts

- **Ambiguity hunt:** teams list five military words whose meaning depends on context (battery, fire, contact, engage, round) and write two sentences for each.
- **Human attention:** for "recon drone spotted enemy armour near ford", each student assigns weights summing to 1 from "armour" to every word; compare with the slide's illustrative weights.

9 Run It Live: Studio Demonstrations

Project `Lecture8_Transformer_Intuition_Lab.html` (it is a single HTML file and runs offline in any browser; every result updates as an input changes). Each demonstration below follows the same rhythm: **set** the inputs, **ask** the class to commit to a prediction (a show of hands or a one-word answer), then **reveal** the result and **land** the point. The numbers are exactly what the page shows for the settings given; simulations use fixed random seeds, so they repeat every time. The 45-minute plan has little slack: choose the one demonstration marked *first choice* for class, and use the others in a tutorial, a second session, or as homework with the drill cards at the bottom of the page.

Demo 1: Temperature and the next word (about 2 min, with §1 Overview)

Set. Demo 1 at the lecture's scores: support 3.1, strike 2.0, cover 1.5, drop -0.5, parade -1.2; $\tau = 1$.

Ask. "If the temperature is halved, what happens to the probability of 'support'?"

Reveal. From 63.5% to **86.8%** at $\tau = 0.5$; at $\tau = 2$ it falls to 43.3%.

Land the point. Softmax turns scores into probabilities; temperature trades predictability for variety. Every candidate keeps some probability.

Demo 2: Context pulls battery (about 4 min, with §6 Attention) — *first choice*

Set. Demo 3, sentence A ("artillery battery fired").

Ask. "Which word will battery attend to most, and where will its vector move?"

Reveal. artillery 0.458, battery 0.196, fired 0.345; $z = (2.40, 0.43)$; contextual battery $(3.40, 1.43)$, cosine 0.946 with artillery and 0.447 with radio. Sentence B: radio 0.480 and battery $(1.43, 3.34)$. Sentence C ("radio battery fired"): $(2.12, 2.71)$, cosines 0.667 and 0.827 — still ambiguous.

Land the point. Same query, same arithmetic, different neighbours: that is all a contextual embedding is.

Demo 3: Why divide by the square root (about 2 min, with §6 Attention)

Set. Demo 4, draw 3, $d_k = 64$.

Ask. "Without scaling, how much weight does the top key get?"

Reveal. 97.3% unscaled against 36.0% scaled; at $d_k = 512$, 100.0% against 27.9%. The raw scores' spread grows like $\sqrt{d_k}$.

Land the point. Scaling keeps softmax out of saturation, so gradients reach W^Q and W^K.

10 Quiz Administration and Answer Key

Read aloud to students

"You will now take a 15-question quiz. Each question has four options and exactly one correct answer. You have 40 seconds per question; the quiz advances automatically when time runs out. A correct answer scores +1, a wrong answer −0.25, and an unanswered question 0. The maximum is 15 and the minimum is −3.75. Because of the negative marking, guessing blindly among four options gains nothing on average — answer when you can eliminate options. Your score and a full review appear at the end."

Delivery. Open `Lecture8_Quiz.html` on each student device (or project it and collect answers on paper). The page enforces the timer and computes the score with negative marking. Release `Lecture8_Answers_Assignment.html` only after all students have finished.

Answer key

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
C	A	D	B	C	A	C	B	D	A	B	C	D	A	B

Interpreting scores. 12 or more: secure command of the lecture. 9–11.75: good, review missed items. Below 9: revisit the reference text before the assignment. If more than half the class misses a question, revisit that point in the next lecture's opening Q&A.

Questions, options and rationale

Q1. The single job a language model is trained to do is to:

A. Look up facts in a database

B. Translate words into ID numbers

C. Predict a probability for every possible next token ✓

D. Classify a sentence as true or false

Rationale: Longer answers come from repeating that job: append the chosen token and predict again.

Q2. Scores $(3.1, 2.0, 1.5, -0.5, -1.2)$ for five candidate words give softmax probabilities of about $(0.635, 0.211, 0.128, 0.017, 0.009)$. What is true of them?

A. They sum to 1 and preserve the order of the scores ✓

B. They sum to 4.9

C. The largest score always gets probability 1

D. Negative scores give negative probabilities

Rationale: Softmax exponentiates and normalises, so all outputs are positive and sum to 1.

Q3. Why are dictionary ID numbers a poor representation of meaning?

A. They are too large for a computer

B. They change every time the model runs

C. They require a GPU

D. Nearby IDs follow spelling or order, not meaning, and arithmetic on them is meaningless ✓

Rationale: *howitzer* = 4,118 sits next to *hospital* = 4,119; embeddings fix this.

Q4. The cosine similarity of the toy vectors artillery $(3, 0.2)$ and radio $(0.2, 3)$ is about:

A. 0.995

B. 0.13 ✓

C. 0.75

D. -1

Rationale: $1.2/(3.007 \times 3.007) = 0.133$: nearly unrelated directions.

Q5. A static embedding of "battery" is a problem because:

A. It has too many dimensions

B. It is learned by backpropagation

C. It gives one fixed vector for every sense of the word, whatever the sentence ✓

D. It cannot be stored in a table

Rationale: The contextual embedding, built by attention, moves the vector towards the meaning intended in that sentence.

Q6. A transformer needs position vectors added to its embeddings because:

A. It processes all tokens in parallel and would otherwise see a bag of words ✓

B. Embeddings are too small

C. Softmax requires them

D. The decoder cannot read text otherwise

Rationale: "Alpha relieves Bravo" and "Bravo relieves Alpha" would be indistinguishable without position.

Q7. In the radio-net analogy, the call-sign each station transmits plays the role of the:

A. Query

B. Value

C. Key ✓

D. Softmax

Rationale: The listener's query is matched against each key; the value is the message passed on.

Q8. Queries, keys and values are produced by:

A. Three separate input sentences

B. Multiplying each token's vector by learned matrices W^Q, W^K, W^V ✓

C. Random sampling at inference

D. Counting word frequencies

Rationale: $q = xW^Q$, $k = xW^K$, $v = xW^V$; the matrices are learned in training and fixed at inference.

Q9. Why are dot-product scores divided by $\sqrt{d_k}$?

A. To make the weights negative

B. To convert them into probabilities

C. To reduce the vocabulary size

D. To stop large scores saturating the softmax and stalling learning ✓

Rationale: Dot products grow with vector length; scaling keeps them in a range where softmax still spreads weight.

Q10. In sentence B, "radio battery died" ($W^Q = W^K = W^V = I$, $d_k = 2$), which word receives the most weight from "battery"?

A. radio, about 0.48 ✓

B. battery, about 0.21

C. died, about 0.31

D. All equal

Rationale: Scores 3.2, 2.0, 2.6 scale to 2.263, 1.414, 1.838; weights 0.480, 0.206, 0.314.

Q11. Multi-head attention in BERT-base uses 12 heads of size 64 because:

A. Each head needs the full 768 numbers

B. $12 \times 64 = 768$**, so the heads together match the embedding width at about the cost of one head** ✓

C. 64 is the vocabulary size

D. Twelve heads are required for English grammar

Rationale: Each head asks its own question in a smaller space; concatenation and W^O restore the width.

Q12. A residual connection in a transformer block:

A. Deletes the original vector

B. Normalises each vector to mean 0

C. Adds the sub-layer's output to its input, so the original survives and gradients flow through the "+" ✓

D. Replaces attention in deep layers

Rationale: Like margin notes on a report: annotations accumulate without retyping.

Q13. Layer-normalising $(2, 4, 6, 8)$ gives approximately:

A. $(0.2, 0.4, 0.6, 0.8)$

B. $(-3, -1, 1, 3)$

C. $(0, 0, 0, 0)$

D. $(-1.34, -0.45, 0.45, 1.34)$ ✓

Rationale: Mean 5, standard deviation $\sqrt{5} = 2.236$: divide the deviations $(-3, -1, 1, 3)$ by 2.236.

Q14. Why does the decoder use a mask during training?

A. To hide future tokens, so each position cannot see the word it must predict ✓

B. To hide the encoder's output

C. To remove rare words

D. To speed up the embedding lookup

Rationale: Masked scores are set to $-\infty$, giving zero weight after softmax.

Q15. In cross-attention, the queries come from the ____ and the keys and values from the ____.

A. encoder; decoder
B. decoder; encoder ✓
C. embedding table; softmax
D. feed-forward network; positional encoding

Rationale: The decoder asks which parts of the input it needs; the encoder's vectors answer.

11 Assignment: Marking Scheme and Model Solutions

Instructions. Five problems, 100 marks, plus a 20-mark bonus problem (Problem 6). Reference: Lecture 8 reference text.

#	Problem	Marks
1	Cosine similarity on a meaning map	20
2	Static versus contextual embeddings	20
3	One attention step by hand	20
4	Encoder and decoder roles; masking	20
5	Assessing a defence use	20
6	Problem 6 (bonus): What two heads add	20
	Total	**100** + 20 bonus

General marking guidance

- Marks for each part are shown in square brackets in the problem statement.
- Award method marks for correct working even when an arithmetic slip carries through; deduct once for the slip, not again for its consequences.
- For interpretive parts, full marks need (i) the correct conclusion, (ii) the reason, and (iii) an operational implication or caveat.
- Answers that state a number without uncertainty, baseline or caveat where one is required lose a mark per part.
- Suggested deadline: one week after the lecture; return marked work before the next quiz on the same topic.

11.1 Problem 1: Cosine similarity on a meaning map (20 marks)

Problem statement

Toy 3-D embeddings: howitzer $(4, 1, 0)$, mortar $(3, 1, 1)$, medic $(0, 1, 4)$, clinic $(0, 2, 4)$, convoy $(2, 3, 1)$.
(a) Compute the cosine similarity of howitzer–mortar, howitzer–medic and medic–clinic. [9]
(b) Compute convoy's similarity to howitzer and to medic, and interpret. [6]
(c) Why is cosine similarity preferred to the raw dot product for comparing meanings? [5]

Model solution and marking points

(a) Norms: howitzer $\sqrt{17} = 4.123$, mortar $\sqrt{11} = 3.317$, medic $\sqrt{17} = 4.123$, clinic $\sqrt{20} = 4.472$. howitzer–mortar $13/(4.123 \times 3.317) = \mathbf{0.951}$; howitzer–medic $1/17 = \mathbf{0.059}$; medic–

clinic $18/(4.123 \times 4.472) = \mathbf{0.976}$.
(b) convoy norm $\sqrt{14} = 3.742$. convoy–howitzer $11/(3.742 \times 4.123) = \mathbf{0.713}$; convoy–medic $7/(3.742 \times 4.123) = \mathbf{0.454}$. A convoy is moderately related to both: it sits between the weapons and medical clusters, closer to the weapons side.
(c) The dot product grows with vector length, which often tracks word frequency rather than meaning. Cosine compares direction only, so two words are judged similar if they point the same way, whatever their lengths.

11.2 Problem 2: Static versus contextual embeddings (20 marks)

Problem statement

(a) Explain, with the word "fire" in "the gunners opened fire" and "the fire spread to the fuel dump", why a single static vector is inadequate. [6]
(b) In a toy 2-D space, gunners $= (2.5, 0.3)$, fuel $= (0.5, 2.5)$ and static fire $= (1, 1)$. Compute the cosine of fire with each, and comment. [6]
(c) What does a contextual embedding add, which part of the transformer produces it, and why can the deciding clue be far away? [8]

Model solution and marking points

(a) The two senses (weapon discharge, blaze) predict different next words and different actions. A static embedding stores one point for both, between the senses, so downstream layers must guess which sense applies and predictions blur.
(b) $\cos(\text{fire}, \text{gunners}) = 2.8/(1.414 \times 2.518) = \mathbf{0.786}$; $\cos(\text{fire}, \text{fuel}) = 3.0/(1.414 \times 2.550) = \mathbf{0.832}$. The static vector is similar to both and decisive for neither.
(c) A contextual embedding is the static vector updated with information gathered from the other tokens of *this* sentence, so "fire" moves towards the weapon or the blaze sense. Attention produces it (with the feed-forward layers refining it). Because attention scores every token against every other directly, a clue nine words or a paragraph back is one step away; a recurrent network would have had to carry it through many steps.

11.3 Problem 3: One attention step by hand (20 marks)

Problem statement

Sentence "fuel fire spread" with toy vectors fuel $(0.5, 2.5)$, fire $(1, 1)$, spread $(0.2, 2.0)$. Take $W^Q = W^K = W^V = I$ and $d_k = 2$.
(a) Compute the scores $q_{\text{fire}} \cdot k$ for the three keys, scale them, and compute the attention weights of "fire". [8]
(b) Compute the attention output z and the residual sum fire $+ z$. [6]
(c) Compute the cosine of the new vector with fuel and with gunners $(2.5, 0.3)$, and compare with Problem 2(b). [6]

Model solution and marking points

(a) Scores: fuel $0.5 + 2.5 = 3.0$; fire 2.0; spread $0.2 + 2.0 = 2.2$. Scaled by $\sqrt{2}$: 2.121, 1.414, 1.556. Exponentials 8.342, 4.113, 4.738, sum 17.193. Weights ($\mathbf{0.485}$, $\mathbf{0.239}$, $\mathbf{0.276}$).
(b) $z = 0.485(0.5, 2.5) + 0.239(1, 1) + 0.276(0.2, 2.0) = (\mathbf{0.537}, \mathbf{2.003})$. Residual: $(1, 1) + z = (\mathbf{1.537}, \mathbf{3.003})$.

(c) With fuel: **0.962** (up from 0.832); with gunners: **0.558** (down from 0.786). One attention step has pulled "fire" firmly towards the blaze sense.

11.4 Problem 4: Encoder and decoder roles; masking (20 marks)

Problem statement

(a) State the job of the encoder and of the decoder in turning "Two enemy tanks moving north near the bridge at 0600" into a SALUTE-style report. [5]

(b) A decoder has unmasked scaled scores $\begin{pmatrix} 2 & 1 & 3 \\ 1 & 2 & 0 \\ 0.5 & 1.5 & 1 \end{pmatrix}$ (row = query position). Apply the causal mask and compute the attention weights. Which row is unchanged, and why? [8]

(c) Explain why the mask is needed in training but the decoder still writes one token at a time at inference. [4]

(d) In cross-attention, where do Q, K and V come from? [3]

Model solution and marking points

(a) The encoder reads the whole report at once and produces a contextual vector for every token. The decoder writes the output one token at a time (SIZE, 2, ACTIVITY, moving, ...), using the encoder's vectors and the tokens already written.

(b) Masked entries above the diagonal become $-\infty$. Row 1: only position 1 visible: $(1, 0, 0)$. Row 2: softmax of $(1, 2)$: $(e^1, e^2)/(e^1 + e^2) = (\mathbf{0.269},\ \mathbf{0.731},\ 0)$. Row 3: nothing is masked, so it is unchanged: (**0.186**, **0.506**, **0.307**). (Unmasked, row 1 would have put 0.665 on the future token 3.)

(c) In training the whole correct output is fed in at once so all positions learn in parallel; without the mask each position could copy the word it must predict. At inference the future does not exist yet, so the decoder generates a token, appends it, and repeats; the mask keeps training and inference consistent.

(d) Queries from the decoder; keys and values from the encoder's output vectors.

11.5 Problem 5: Assessing a defence use (20 marks)

Problem statement

A unit proposes a transformer that turns patrol voice-note transcripts into structured SALUTE reports for the operations room.

(a) Give two benefits. [4]

(b) Give four risks, each with a concrete safeguard. [10]

(c) If each of the six fields is independently wrong 3% of the time, what share of reports contain at least one wrong field? What error rate per field is needed to bring that below 6%? [6]

Model solution and marking points

(a) Speed and consistency under load; resolution of context-dependent terms ("contact", "battery", "fire"); searchable, structured records for later analysis.

(b) **Hallucination** (invented units, grids or times): show the source words each field came from (cross-attention alignment) and require human confirmation. **Jargon and grid references** split into many tokens: fine-tune on unit vocabulary and validate grids against a format check. **Prompt injection** in transcribed text: treat input as data, strip instructions, restrict outputs

to the schema. **Over-trust in explanations**: attention maps are clues, not proof; audit a sample of reports against the audio.
(c) $1 - 0.97^6 = 1 - 0.833 = \mathbf{16.7}\%$ of reports have at least one error. Need $1 - (1-e)^6 < 0.06$, i.e. $(1-e)^6 > 0.94$, $1 - e > 0.9897$, $e < \mathbf{1.03}\%$; at $e = 1\%$ the share is 5.9%.

11.6 Problem 6 (bonus): What two heads add (20 marks)

Problem statement

(a) Explain why one attention head can mainly capture one kind of relationship at a time. [4]
(b) Two heads produce, for the token "armour", $z_1 = (0.8, 0.1)$ (a modifier head drawing on "enemy") and $z_2 = (0.2, 0.9)$ (a location head drawing on "ford"). Form the concatenation and apply $W^O = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$ (as $[z_1; z_2]W^O$). [6]
(c) Show that a multi-head layer with h heads of size d/h has the same number of projection weights as one head of size d, and compute it for BERT-base. [10]

Model solution and marking points

(a) Each head produces one set of weights per token, a single softmax over the sentence. If "armour" must learn both *whose* it is and *where* it is, one distribution has to compromise between "enemy" and "ford"; separate heads can each attend sharply.
(b) $[z_1; z_2] = (0.8, 0.1, 0.2, 0.9)$. Column j of W^O picks entry: output $= (0.8,\ 0.2,\ 0.1,\ 0.9)$. W^O here simply interleaves the heads; a trained W^O mixes them into combinations useful to later layers.
(c) Each head has W_i^Q, W_i^K, W_i^V of size $d \times d/h$: $3d^2/h$ weights, so h heads have $3d^2$, the same as one head of width d; W^O adds d^2. BERT-base: $4 \times 768^2 = 2{,}359{,}296$ weights, plus $4 \times 768 = 3{,}072$ biases, 2,362,368 in all, whether $h = 1$ or 12.